

DEVOPS / CLOUD ENGINEER

김준우 Joon Woo Kim

사람과 환경에 대한 관심에서 시작해,
제작 시스템을 거쳐 인프라와 자동화를 만들고 있습니다.

joon@junu.dev
junu.dev
github.com/Vivacissimo
wiki.junu.dev
Portfolio · resume.junu.dev

01
INTRODUCE

현재는 Cloud Native Infrastructure와 DevOps에 관심을 두고, Kubernetes 기반 서비스 운영과 CI/CD, IaC, Observability를 중심으로 경험을 넓히고 있습니다.

사회복지에서 출발해 VFX 분야로 옮긴 뒤, 결과물보다 그것이 만들어지는 **환경과 시스템**에 더 큰 흥미를 느꼈습니다. Render Farm과 Linux 제작 환경을 운영하며 반복 작업을 자동화했고, 관심사는 **Pipeline Automation → IT Infrastructure → Cloud / DevOps**로 이어졌습니다.

> 반복 가능하고, 관찰 가능하며, 운영할 수 있는 시스템을 만드는 엔지니어를 지향합니다.

02
EXPERIENCE

KT Cloud BootCamp

2026.03 — 2026.10 (수료)

교육 과정 · 팀 프로젝트

KT Bootcamp의 Cloud-Native 과정을 수료하며 진행했던 프로젝트입니다.

CLOUD / DEVOPS · INTEGRATED PROJECT

MoongCheap · Cloud Native Platform

CI/CD · Kubernetes · GitOps 담당 | 팀 프로젝트 대상 및 우수 수료자 선정

PROBLEM
서비스별 YAML 중복, dev/prod 설정 혼재, 애플리케이션 코드와 배포 설정의 책임 경계가 불명확했다.

ACTION
Jenkins → ECR → GitOps → ArgoCD → EKS 흐름 설계, 공통 Helm Chart + 환경별 Values, ApplicationSet, Commit SHA 기반 Image Tag.

RESULT
Frontend · Backend · AI 3개 영역과 dev/prod 환경을 공통 Chart와 ApplicationSet 규칙 하나로 배포하는 구조를 코드로 정리 함.

AWS · EKS · ECR · Helm · ArgoCD · Jenkins · Terraform

KUBERNETES · CI/CD · GITOPS · OBSERVABILITY

Hotel Booking · GitOps & Observability

3인 팀 · Team Lead · Infra & CI/CD · 2026.07

PROBLEM
특가 오픈 순간 트래픽이 몰리는 5개 서비스 MSA를, 수동 배포 없이 dev/prod에 일관되게 올리고 장애를 빠르게 감지해야 했다.

ACTION
Terraform으로 EKS · ArgoCD 구성, GitHub Actions 변경 감지 Matrix 빌드 + Trivy 스캔 + ECR Push, SHA Tag로 Helm values 갱신 → ArgoCD 자동 Sync, prod 매니페스트 검증.

RESULT
push 한 번으로 5개 서비스를 dev/prod에 배포. 그 위에서 K6 1,000명 버스트 오버셀 0건 · 시스템 오류 0%, 장애 주입 → Slack 알림 · 복구 확인.

AWS EKS · Terraform · GitHub Actions · Trivy · Helm · ArgoCD · Prometheus · Loki · Tempo

CI/CD · GITOPS · SERVICE MESH · OBSERVABILITY

Medical MSA · CI/CD & Service Mesh

4인 팀 · 기본 CI/CD · GitOps · K8s 운영 → 심화 Team Lead · Observability · 2026.05 – 06

PROBLEM
수동 배포로 휴먼 에러와 서비스 중단 위험이 컸고, 배포가 성공해도 새 버전이 트래픽과 장애 상황에서 정상인지 판단할 기준이 없었다.

ACTION
GitHub Actions 변경 감지 Matrix 빌드 · 멀티스테이지 이미지 · Trivy 차단 · git-sha Tag로 Helm values 갱신 → ArgoCD dev/prod 배포, HPA · Probe · Rolling Update 구성과 장애 유도 검증. 심화에서 Istio Canary · Blue/Green · Retry · mTLS 실험과 Prometheus · Grafana · Kiali 장애 추적 Runbook 정리.

RESULT
4개 서비스를 push 한 번으로 자동 배포. Pod 강제 종료 중 클라이언트 성공률 100%, 비인가 Pod 접근 차단 (RBAC denied)을 검증.

GitHub Actions · Trivy · Helm · ArgoCD · AWS EKS · Istio · Prometheus · Grafana · Kiali

Cocoavision

2022.09 — 2026.03

VFX본부 · Pipeline / IT Infrastructure
2024.01 — 2026.03

NewMedia본부 R&D팀
2022.09 — 2024.01

실시간 콘텐츠 R&D팀에서 시작해 VFX본부로 이동한 뒤, Linux 제작 환경과 Render Farm, Pipeline 운영을 담당했다.

VFX / AUTOMATION

Render Farm Automation with Ansible

PROBLEM
수십 대 규모 Linux Render Farm을 노드별로 수동 설정하며 반복 작업과 설정 편차가 발생했다.

ACTION
패키지 설치 · Mount · 팀별 Tool 배포 등을 Ansible Playbook으로 작성하고 Inventory로 노드를 묶어 일괄 적용.

RESULT
서버별 수동 작업을 재사용 가능한 운영 코드로 전환.

Ansible · Linux · Render Farm

VFX / INFRASTRUCTURE

PXE 기반 네트워크 설치 환경

PROBLEM
장비가 늘어날수록 USB 중심의 PC 초기 설치를 사람이 직접 반복해야 했다.

ACTION
PXE 부팅부터 DHCP/HTTP 설치 파일 제공, Kickstart 기반 자동 설치까지 연결한 환경 구성.

RESULT
USB 중심 설치를 네트워크 기반 설치로 전환.

Linux · PXE · Kickstart · Network Boot

VFX / PIPELINE

Tractor Job Submission 자동화

PROBLEM
작업자가 제출 정보를 직접 구성하고 DCC마다 절차가 달라 누락과 설정 차이가 발생하기 쉬웠다.

ACTION
Houdini · Nuke용 Custom Render / Submit Node를 Python으로 만들어, 사용자 교육 대신 도구에 제출 규칙을 넣음.

RESULT
DCC 안에서 일관된 제출 경로를 제공해 Submission Workflow 표준화.

Houdini · Nuke · Tractor · Python

03 SKILLS

Cloud
AWS
EC2 · VPC · EKS · ECR · IAM · S3

CI/CD & GitOps
Jenkins · GitHub Actions · ArgoCD
Pipeline · Webhook · Self-hosted Runner · ApplicationSet · Sync

IaC & Configuration
Terraform · Ansible
Provider · Resource · Module · State · Inventory · Playbook

Scripting
Python
DCC Tool · Render Submission · Pipeline Automation

Container & Orchestration
Docker · Kubernetes · Helm
Image · Docker Compose · Pod · Deployment · Service · Ingress · Chart

Service Mesh & Observability
Istio · Prometheus · Grafana · Kiali
Canary · Blue/Green · mTLS · Metric · Dashboard · Service Topology

Systems & Network
Linux · Nginx · Cloudflare · PXE · Git
Reverse Proxy · DNS · Cloudflare Tunnel · Network Boot · Kickstart · Version Control

VFX Pipeline
Houdini · Nuke · Maya · Tractor
Production Pipeline · Render Farm · Troubleshooting

04 PERSONAL

PERSONAL INFRASTRUCTURE

junu.dev Home Infrastructure

Raspberry Pi · Docker Compose 기반으로 Wiki와 개인 서비스를 직접 운영하고, Cloudflare Tunnel과 GitHub Actions Self-hosted Runner로 외부 연결과 배포를 자동화.

Linux · Docker · Nginx · Cloudflare Tunnel · GitHub Actions · PostgreSQL

Portfolio ↗ Wiki ↗

PERSONAL DEV TOOLS

Personal Dev Tools

개인 작업에서 반복되던 이미지 편집과 파일 확장자 변환을 독립 웹 도구로 만들어 서비스별 Subdomain으로 운영.

AI Agent · Web Tool · Nginx · Cloudflare

Maskly ↗ Transformer ↗

KT CLOUD TECH UP · 통합 프로젝트 · 팀 프로젝트 대상

MoongCheap · AI 역경매 공동구매 플랫폼

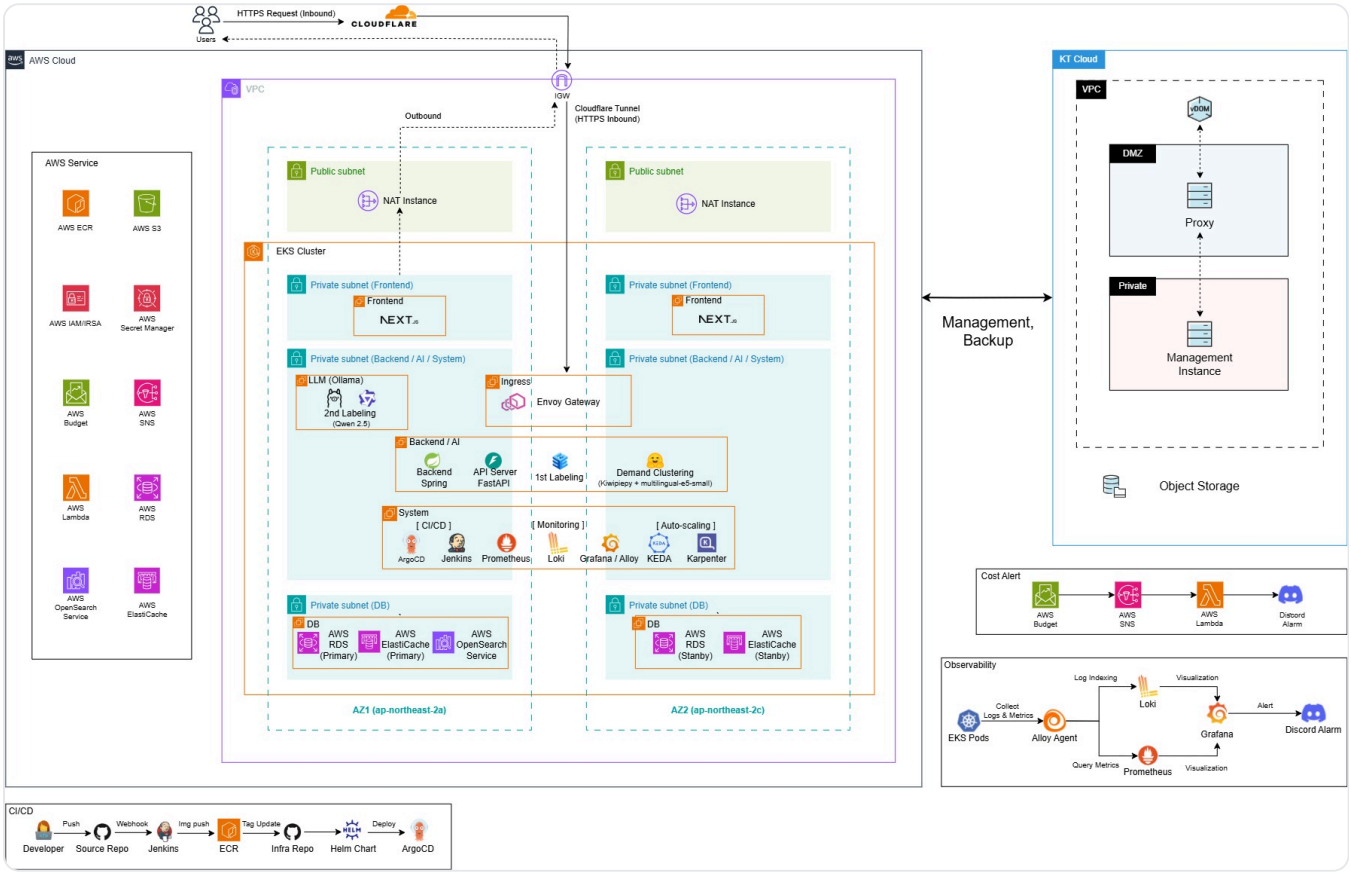
01

AWS EKS Multi-AZ 위의 FE · BE · AI 서비스를 공통 Chart와 GitOps 규칙 하나로 배포하는 구조를 만들고, 운영 중 생긴 병목을 실측으로 해결했다.

상세: resume.junu.dev/projects/moongcheap

PERIOD	TEAM	ROLE	STACK
2026.08 – 10	Cloud 파트 5인	CI/CD · GitOps · AI 배포	EKS · Jenkins · ECR · Helm · ArgoCD

Kubernetes 운영 · CI/CD · GitOps · 장애 원인 분석 · AWS EKS



내가 한 일

- Jenkins → ECR → GitOps Repo → ArgoCD ApplicationSet 흐름 설계, develop / prod / platform 별 Application 자동 생성
- FE · BE · AI가 **moongcheap-service** Chart 하나를 쓰고, 차이는 base → env → services → overrides 4단계 Values로만 관리
- Commit SHA 이미지 태그로 운영 중인 Pod가 어떤 커밋인지 바로 추적
- App Repo와 GitOps Repo를 분리하고, 서비스 팀이 넘길 항목(Port · Health Check · 환경 변수)을 정의해 배포 책임 경계를 정리

- AI 팀의 CPU · GPU · Memory · 추론 API 요구사항을 받아 같은 GitOps 흐름으로 AI · LLM 서비스 배포

트러블슈팅

노드에 여유가 있는데 ArgoCD · Jenkins가 Pending

노드당 Pod 한도 17개로 **Too many pods** → VPC CNI Prefix Delegation · **maxPods: 110**을 Close 시간에 무중단 적용
17 → 110개

인프라를 켠 뒤 CI 서버가 1시간 반 넘게 안 열림

기동마다 플러그인 100개 재설치 → **initializeOnce: true** 한 줄 **96분 → 2.6분**

백엔드 빌드 1회 45분

빌드 Pod 메모리 과다 요청으로 빌드마다 노드 4번 추가 → Pod 46개 실측으로 요청값 재산정 **45분 → 16분**

CI 기동
97% 단축
96분 이상 → 2.6분

BUILD
64% 단축
노드 추가 4회 → 1회

POD LIMIT
6.5배
노드 추가 비용 없이

KT CLOUD TECH UP · 실무 프로젝트

Hotel Booking · GitOps & Observability

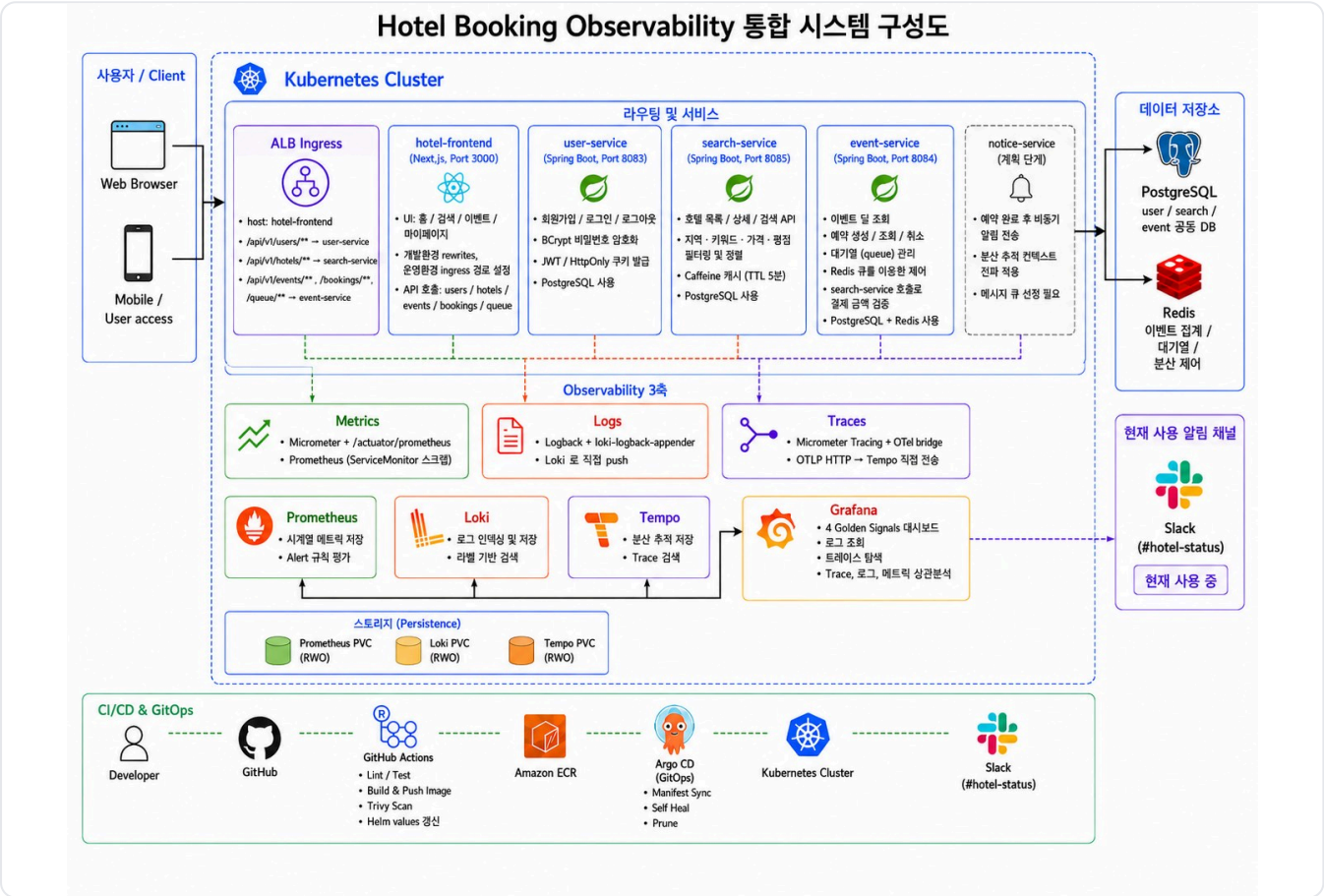
02

특가 오픈 순간 트래픽이 몰리는 호텔 예약 MSA를 Terraform으로 만든 EKS에 올리고, push 한 번으로 dev/prod까지 이어지는 배포 흐름을 만들었다.

상세: resume.junu.dev/projects/hotel-booking

PERIOD	TEAM	ROLE	STACK
2026.07	3인	팀장 · 인프라 & CI/CD	EKS · Terraform · GitHub Actions · ArgoCD

IaC (Terraform) · CI/CD · GitOps · 모니터링 · 알림 · AWS EKS



내가 한 일

- **terraform-aws-modules/eks**로 노드 그룹 · IRSA · Access Entry를 구성하고 ArgoCD도 Terraform Helm Release로 설치
- GitHub Actions에서 변경된 서비스만 Matrix 빌드 → Gradle 테스트 → Trivy HIGH/CRITICAL 차단 → ECR Push
- CI가 Helm values의 이미지 태그를 커밋하면 ArgoCD(App of Apps · prune · selfHeal)가 dev/prod에 자동 배포
- prod 파이프라인에 **helm lint**와 렌더링 검사를 넣어 dev 네임스페이스 · 브랜치 설정 유입을 차단
- 공통 Chart에 HPA(2–5) · 무중단 Rolling Update · ALB Ingress 경로 라우팅 적용

트러블슈팅

ALB에서 로그인 후 401 반복

HTTP ALB에서 Secure 쿠키가 저장되지 않음 → 쿠키 속성을 환경 변수로 분리해 환경별 values로 주입

배포 직후 Pod 반복 재시작

초기화 전에 Liveness Probe가 실패 판정 → 공통값 **initialDelaySeconds** 45초 · **failureThreshold** 6

클러스터에서만 API 호출 실패

CORS 분산 · 로컬 주소 직접 호출 → CORS 공통화, 상대경로 호출, Ingress **/api/v1/*** 경로 라우팅

LOAD TEST

오버셀 0건

K6 1,000명 · 재고 30

ERROR

시스템 오류 0%

매진(409)은 정상 응답으로 분리

DEPLOY

5개 서비스 · 2개 환경

push 한 번으로 자동 배포

KT CLOUD TECH UP · 기본 + 심화 프로젝트

Medical MSA · CI/CD & Service Mesh

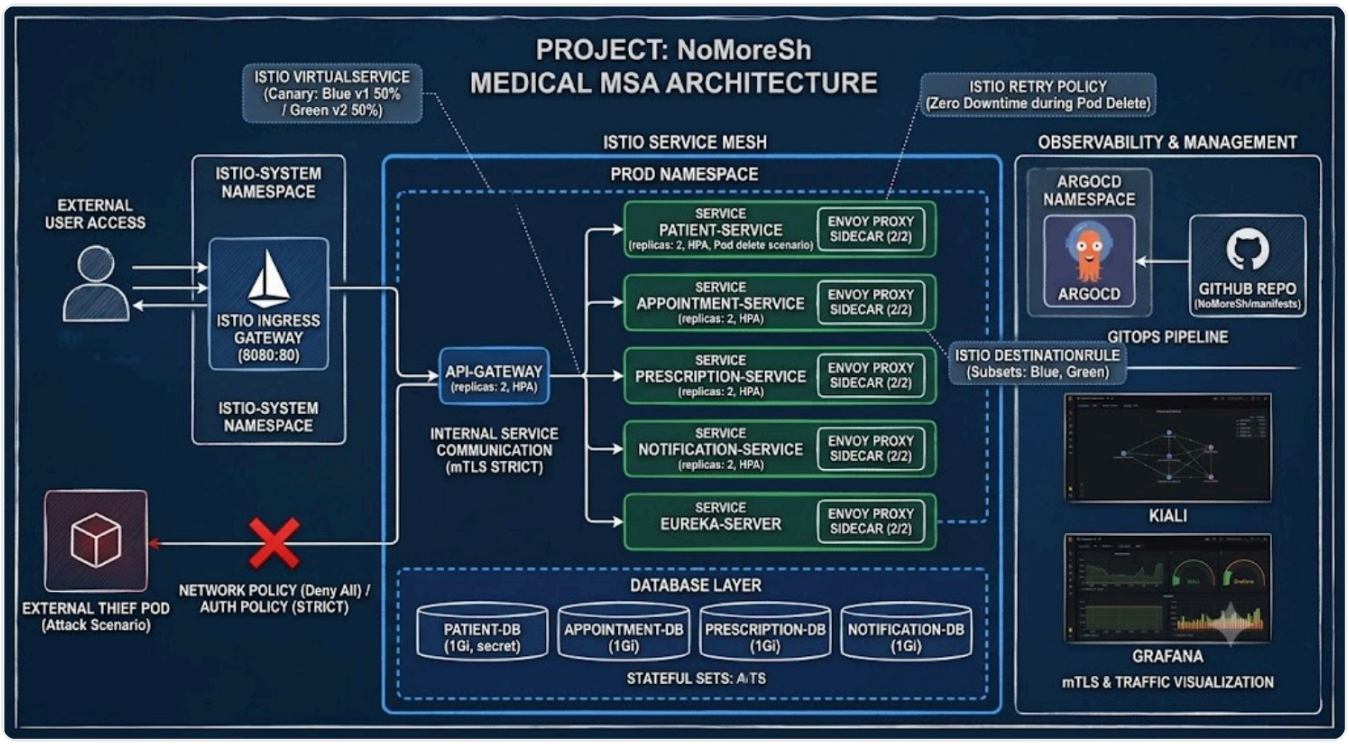
03

의료 MSA의 CI/CD · GitOps 배포를 자동화하고, 심화 단계에서 Istio를 올려 트래픽 전환 · 장애 복구 · 내부 접근 차단을 직접 실험했다.

상세: resume.junu.dev/projects/cloud-native

PERIOD	TEAM	ROLE	STACK
2026.05 – 06	4인	CI/CD · K8s 운영 → 심화 팀장	GitHub Actions · ArgoCD · EKS · Istio

Kubernetes 운영 · Service Mesh (Istio) · CI/CD · 모니터링



내가 한 일

- Path Filter로 변경된 서비스만 Matrix 빌드, 테스트 실패 시 중단
- jdk → jre 멀티스테이지 · 비루트 실행 이미지, Trivy **exit-code 1**로 취약 이미지 레지스트리 유입 차단
- git-sha 태그 Push → Helm values 갱신 → ArgoCD **medical-dev** / **medical-prod** 분리 배포, 결과는 Slack으로 공유
- HPA(CPU 70%, 2–10) · Readiness/Liveness Probe · Rolling Update 구성, DB 장애 · Memory Leak 유도로 Probe 동작 검증
- 심화 팀장: Prometheus로 K8s · Envoy 메트릭 수집, Grafana → Kiali → ArgoCD 장애 추적 Runbook 정리

검증 · 트러블슈팅

Canary · Blue/Green

VirtualService 가중치로 20 → 50 → 70 → 100% 단계 전환, 50:50 → 0:100 즉시 전환

Pod 강제 종료 중 재시도

retries.attempts: 3 적용 후 **kubectl delete pod** → Envoy가 살아 있는 Pod로 재시도 **성공률 100%**

Zero-Trust 접근 차단

mTLS STRICT · AuthorizationPolicy 상태에서 Mesh 밖 Pod의 직접 호출 → **RBAC: access denied**

빌드 아키텍처 차이로 배포 실패

QEMU · Docker Buildx 멀티 플랫폼 빌드로 해결

CI/CD
4개 서비스 자동 배포
Trivy 통과 이미지만 반영

RETRY
성공률 100%
Pod 강제 종료 중

CANARY
4단계 전환
20 → 50 → 70 → 100%

PROFESSIONAL EXPERIENCE · 2022.09 – 2026.03

Cocoavision · VFX Pipeline / Infrastructure

04

아티스트가 쓰는 DCC 환경과 Linux Render Farm을 운영하며, 반복되는 설치 · 설정 · 제출 작업을 자동화하고 제작이 멈추는 원인을 해결했다.

상세: resume.junu.dev/projects/cocoavision

PERIOD	TEAM	ROLE	STACK
2022.09 – 2026.03	VFX본부 · NewMedia R&D팀	Pipeline / IT Infrastructure	Linux · Ansible · PXE · Python · Tractor

IaC (Ansible) · Linux 서버 운영 · Python 자동화 도구 · 장애 원인 분석

Case 1 · Ansible Render Farm

- **문제** 수십 대 Render Node에 SSH로 접속해 패키지 · Mount · 팀별 Tool을 노드마다 수동 설정, 설정 편차 발생
- **해결** Inventory로 노드를 그룹화하고 반복 작업을 Playbook으로 선언해 일괄 적용
- **결과** 서버별 수동 작업을 재사용 가능한 운영 코드로 전환

Case 2 · PXE 네트워크 설치

- **문제** USB로 장비마다 OS를 설치해, 장비가 늘수록 사람이 하는 단계가 증가
- **해결** PXE 부팅 · DHCP/HTTP 설치 파일 제공 · Kickstart 자동 설치를 연결
- **결과** 초기 세팅을 같은 절차로 반복할 수 있는 네트워크 설치로 전환

Case 3 · Tractor 제출 도구

- **문제** 작업자가 렌더 제출 정보를 직접 구성하고 DCC마다 절차가 달라 누락 · 설정 차이 발생
- **해결** Houdini · Nuke용 Custom Submit Node를 Python으로 개발. 사용자 교육 대신 제출 규칙을 도구에 넣음
- **결과** DCC 안에서 일관된 제출 경로를 제공해 Submission Workflow 표준화

장애 대응 · Farm에서만 실패하는 렌더

- 로컬에선 열리지만 Farm 제출 후 반복 실패해 제작이 중단
- Tractor Log → DCC/Plugin 버전 → 환경 변수 → 파일 경로 → Mount 순으로 실행 환경 차이를 비교해 원인 특정
- 설정 수정 후 같은 유형이 다른 노드에서 반복되지 않도록 운영 절차와 자동화 대상에 반영

FARM	AUTOMATION	WORK STYLE
수십 대 노드	**3개 영역 자동화**	**Fix → Standardize → Automate**
Linux Render Node 운영	Submit · Provisioning · Configuration	반복 이슈를 구조로 줄임